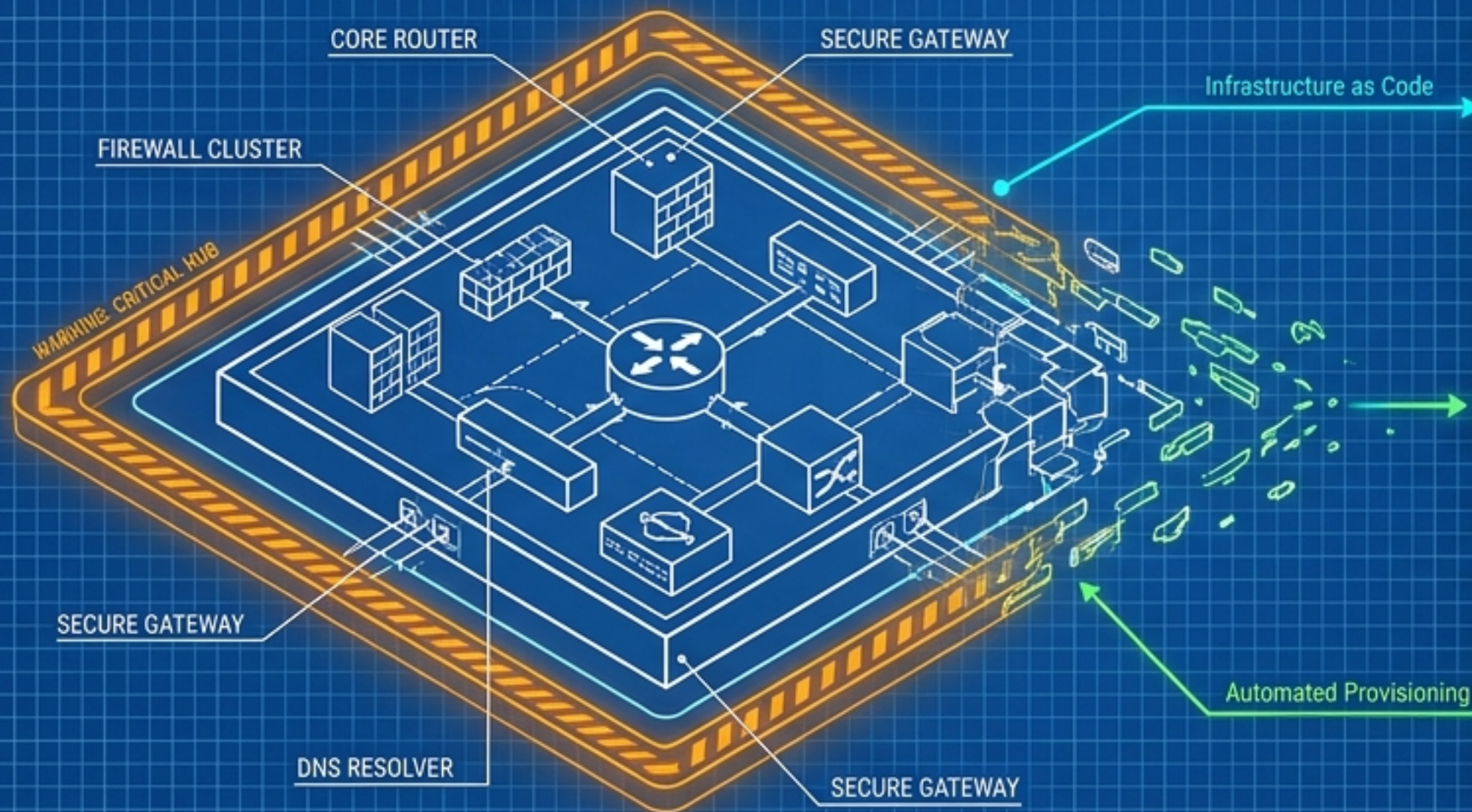


ENTERPRISE CLOUD INFRASTRUCTURE BASELINE

ORCHESTRATING NETWORKING AND DNS THROUGH TERRAFORM: FROM TOPOLOGY TO CODE



```
# Central Network Hub Configuration
resource "aws_vpc" "main" {
  cidr_block      = "10.0.0.0/16"
  enable_dns_support = true
  enable_dns_hostnames = true
  tags = {
    Name = "Enterprise-Core-VPC"
    Environment = "Production"
  }
}

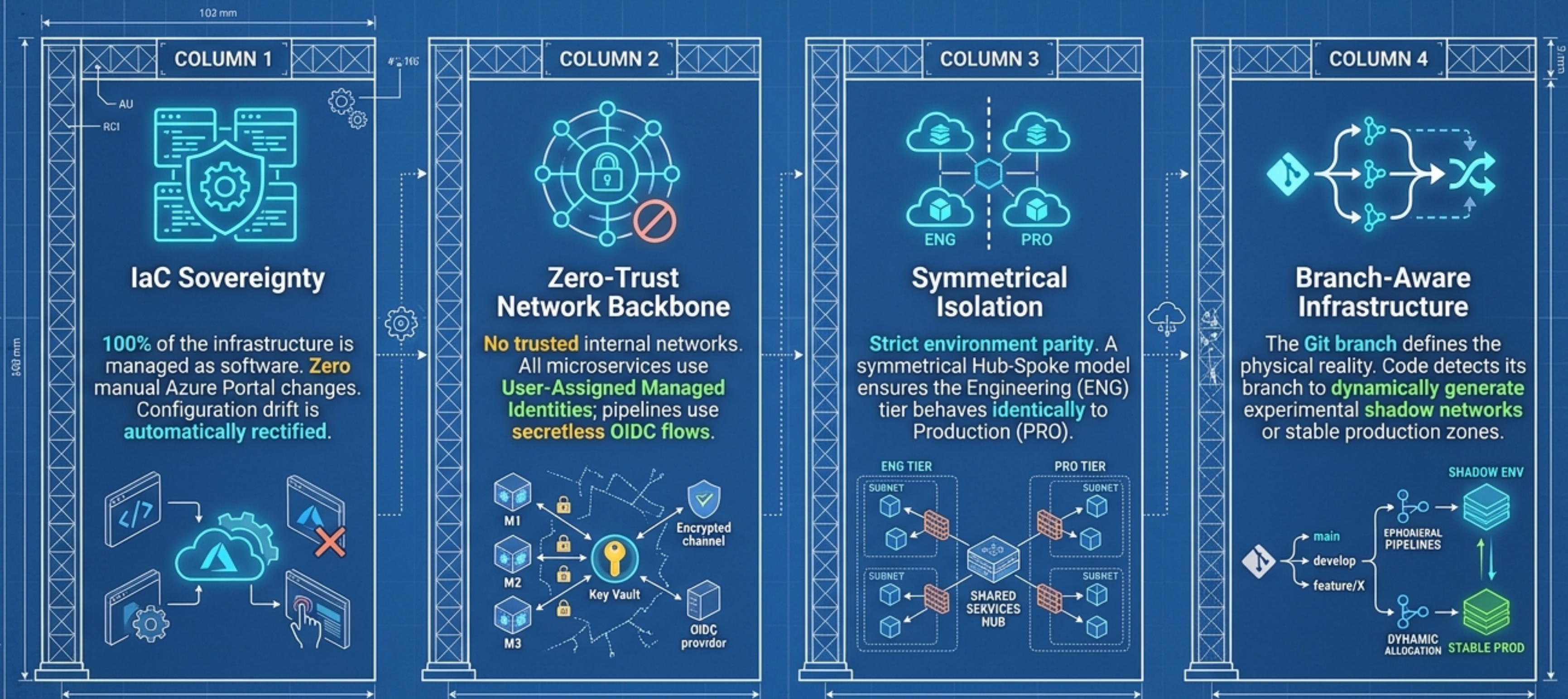
resource "aws_subnet" "public" {
  vpc_id            = aws_vpc.main.id
  cidr_block        = "10.0.1.0/24"
  map_public_ip_on_launch = true
  availability_zone  = "us-east-1a"
  tags = {
    Name = "Public-Subnet-A"
  }
}

resource "aws_route53_zone" "primary" {
  name = "enterprise.internal"
}

# Security Group for Core Services
resource "aws_security_group" "core_sg" {
  name        = "core-services-sg"
  description = "Allow internet traffic"
  vpc_id      = aws_vpc.main.id

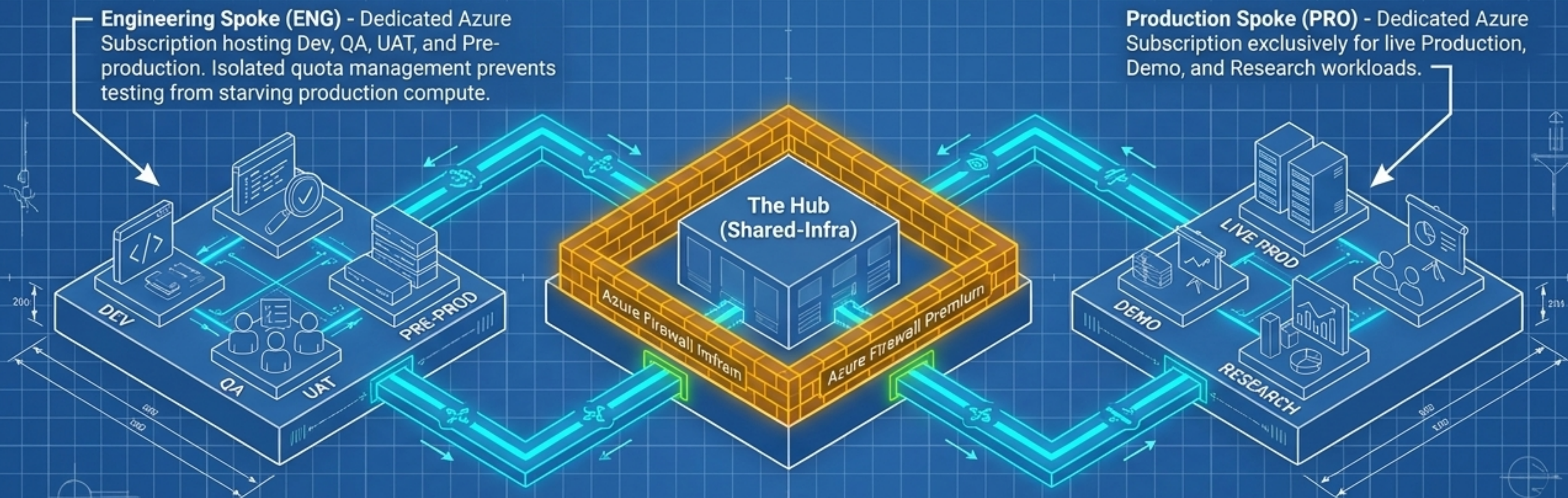
  ingress {
    from_port = 0
    to_port   = 0
    protocol  = "-1"
    cidr_blocks = ["10.0.0.0/16"]
  }
}
```


THE FOUNDATIONAL ENGINEERING TENETS



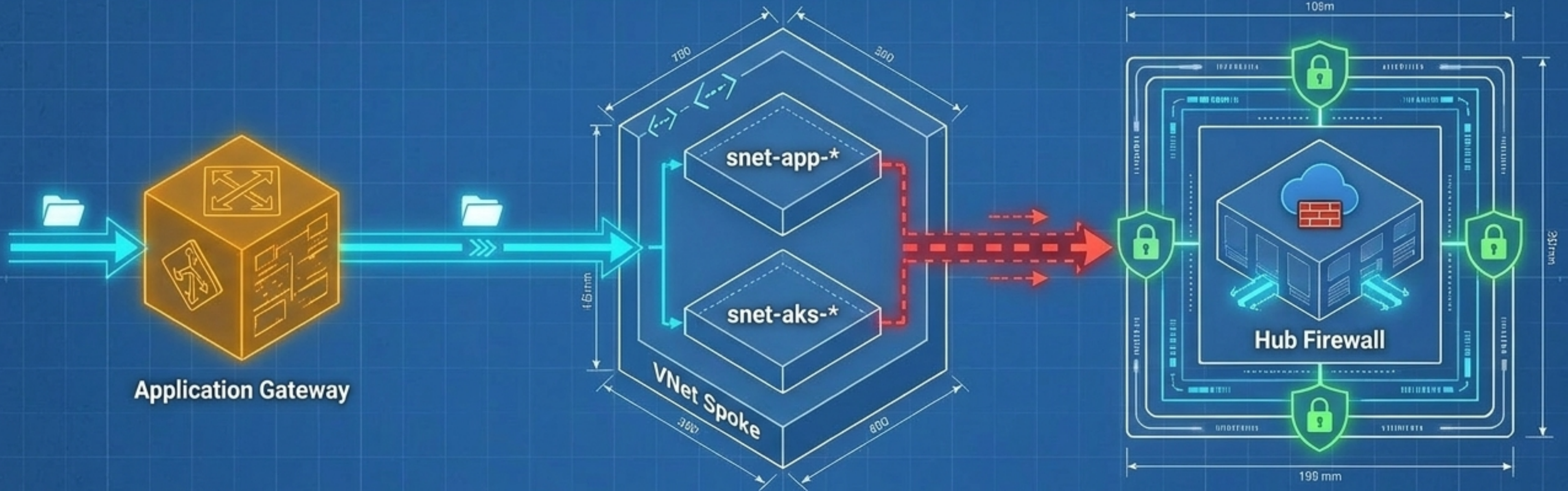
The Dynamic Blueprint

Symmetrical Hub-and-Spoke Topology



Core Insight: By mirroring the topology across distinct subscription spokes, we achieve absolute billing isolation and eliminate noisy neighbor risks during heavy load testing.

Zero-Trust Traffic Engineering and Edge Security



Step 1: Edge Termination

Application Gateway WAF v2 terminates SSL/TLS at the edge, inspecting L7 traffic before it ever touches a workload.

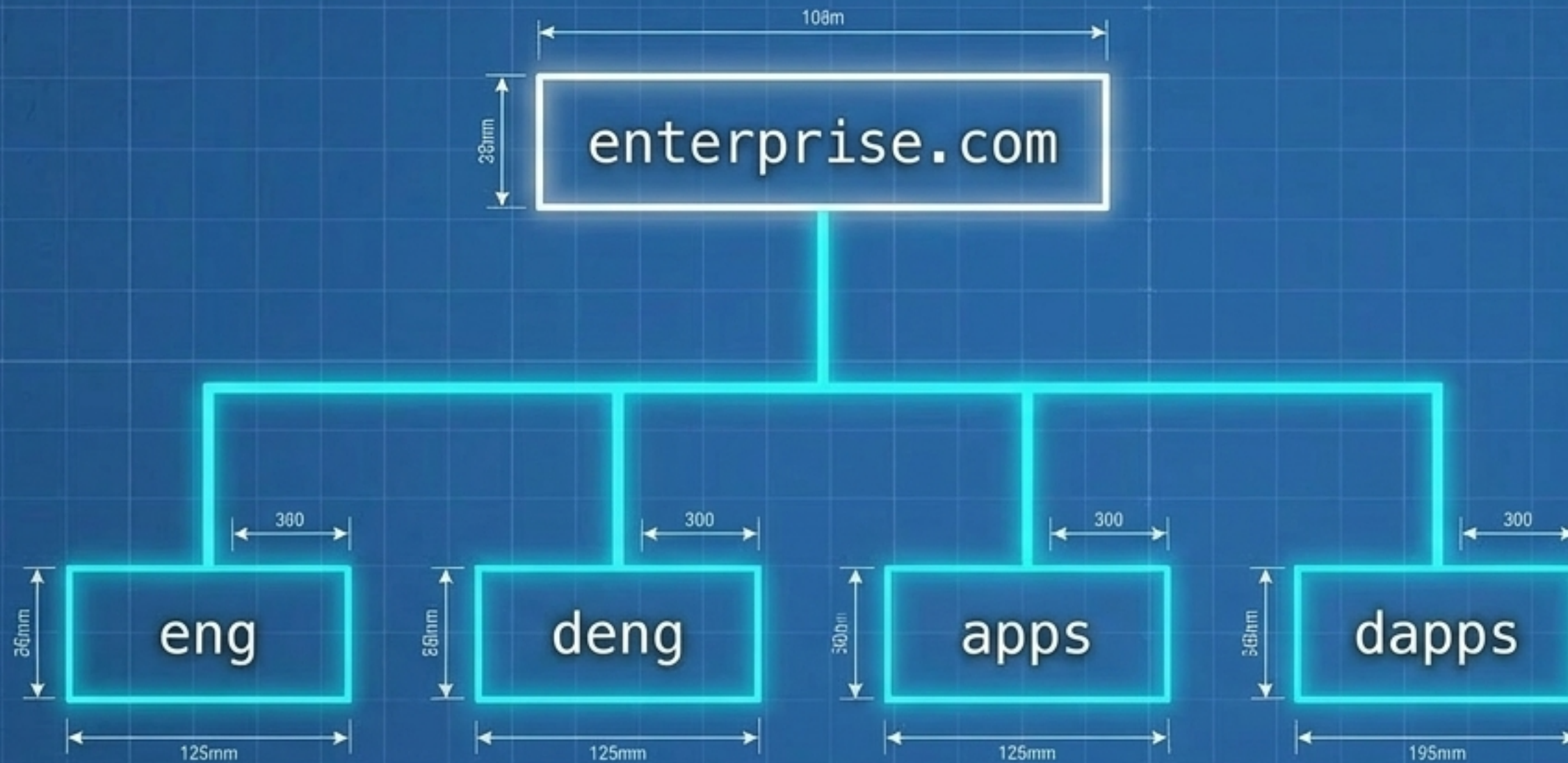
Step 2: Spoke Peering

Traffic securely traverses VNet peering links into the highly restricted workload subnets.

Step 3: Forced Tunneling

User Defined Routes (UDRs) are attached via Terraform (0.0.0.0/0). Egress traffic is strictly forced through the Azure Firewall Premium in the Hub for deep packet inspection.

The Private DNS Orchestration Strategy

**Why do we need multiple DNS zones per environment?**

To prevent namespace collisions. Platform engineers actively testing new Terraform modules in a laboratory branch cannot risk overwriting the DNS records of stable, business-facing test environments.

The architecture requires a Shadow Ecosystem that runs entirely in parallel to the stable ecosystem.

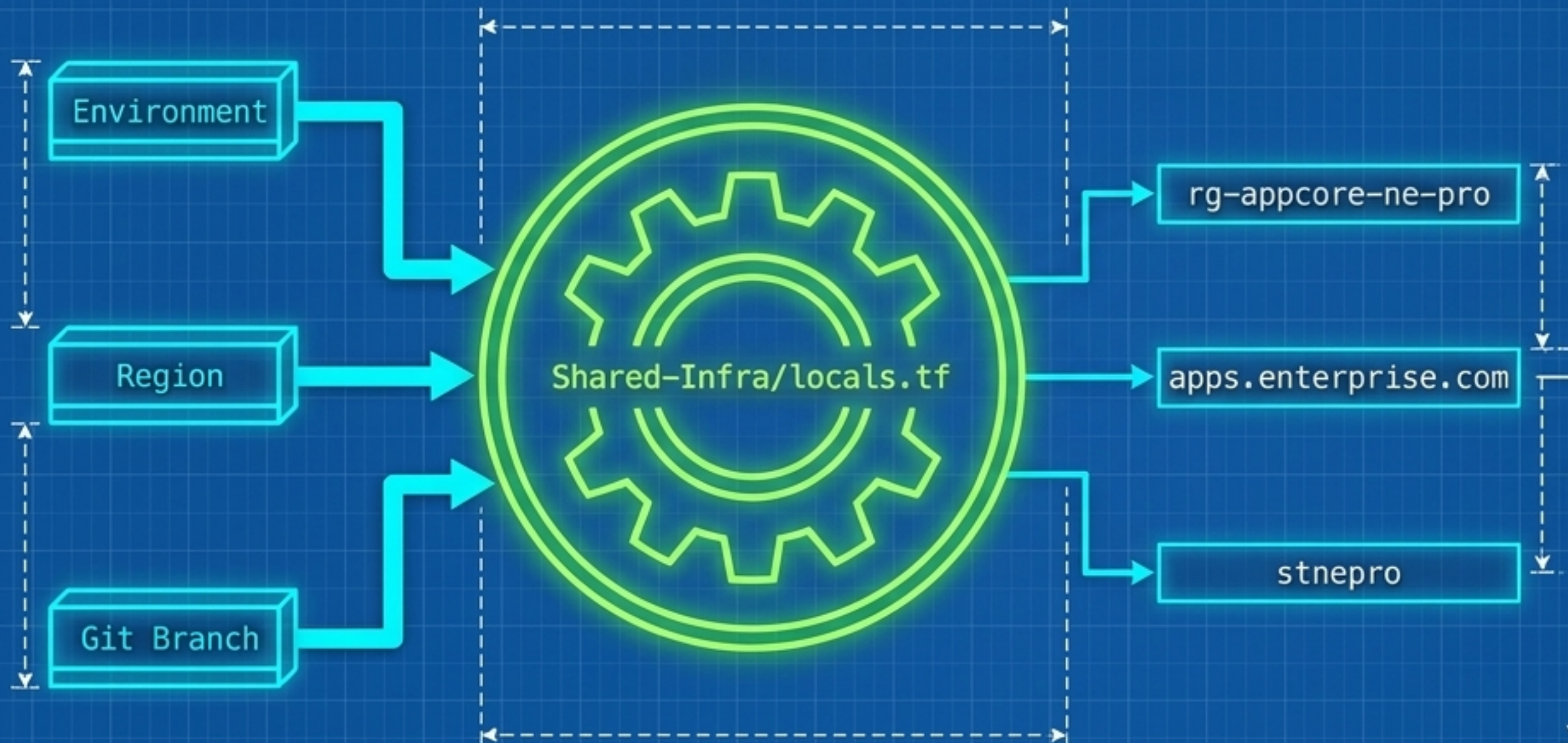
The DNS Environment Matrix

		Application Tier	
		Engineering Spoke	Production Spoke
Git Branch	main	<code>eng.enterprise.com</code> Stable IaC hosting Dev, QA, UAT, Pre.	<code>apps.enterprise.com</code> Stable IaC hosting live Pro, Dem, Res.
	develop	<code>deng.enterprise.com</code> Experimental IaC shadow testing for Dev, QA, UAT.	<code>dapps.enterprise.com</code> Experimental IaC shadow testing for Pro, Dem.

Notice the **d** prefix. The **d** mathematically denotes any infrastructure provisioned from a non-main development branch.

The Dynamic Blueprint

The DRY Logic Engine: Infrastructure DNA



The Problem:

Hardcoding resource names leads to drift, collision, and copy-paste errors across dozens of .tfvars files.

The Solution:

A multi-layer custom logic engine inside locals.tf. It ingests pure intent (e.g., Deploy Dev to Europe from a feature branch) and automatically calculates the exact nomenclature, global tags, and DNS zone placement.

The Result:

Hundreds of Azure resources are named and routed deterministically without duplicating a single line of Terraform code.

PROJECT:

CLOUD_ARCHITECTURE_V2.3

DRAWING NO:

F-ET-004

DATE:

OCT 28 2024

Dynamic FQDN and Nomenclature Generation

Code Editor

```
# Branch Awareness Logic
gitbranch = (var.gitbranch != "main") ? "d" : ""

# DNS Name Calculation
appcore_dns_name = "${dns_prefix}${gitbranch}
{env}.enterprise.com"
```

Top Transformation Panel

Input: Branch = develop, Env = nedev, Tier = ENG

Calculation: Prefix becomes deng

Output: appcore-dnedev.deng.enterprise.com
(Shadow Infrastructure)

Bottom Transformation Panel

Input: Branch = main, Env = nepro, Tier = PRO

Calculation: Prefix is empty

Output: appcore-nepro.apps.enterprise.com
(Immutable Production)

The Dynamic Blueprint

Secretless TLS and Certificate Management



1. Central Vaulting

Wildcard certificates (*.eng.enterprise.com, *.apps.enterprise.com) are securely stored in a dedicated **Shared-Infra Key Vault**.

2. Dynamic Provisioning

The Terraform App Gateway module never handles the physical .pfx file. It dynamically references the Key Vault Secret ID.

```
app-core_agw.ssl_certificate.key_vault_secret_id
```

3. Day-2 Rotation

Because certificates are referenced via versionless URLs, Azure automatically rotates the binding on the App Gateway when the Key Vault certificate is renewed, requiring zero pipeline executions.

Global Fan-Out and Feature Flags



Feature Flags

Regional deployments are entirely toggleable via `.tfvars` boolean flags (`deploy_Europe = true`, `deploy_United_States = true`).

Regional Nomenclature

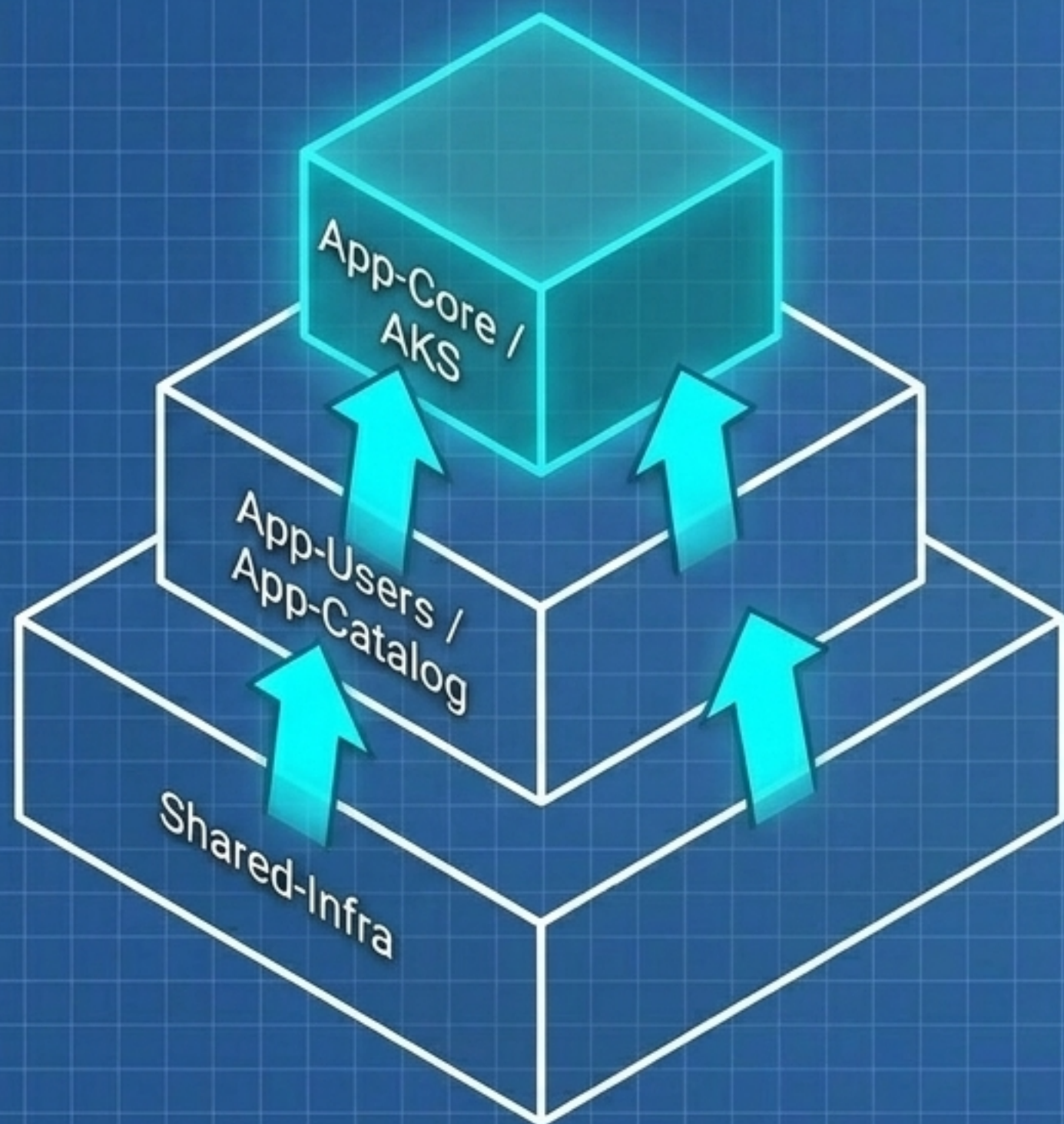
The Logic Engine seamlessly injects regional short-codes (`ne` for North Europe, `us` for East US) into the FQDNs to prevent global namespace overlap.

Resulting Endpoints

- `app-core-nedev.eng.enterprise.com`
- `app-core-usdev.eng.enterprise.com`

Benefit: Deploying a completely new continental footprint requires changing exactly one word from false to true.

Pipeline Architecture & Referential Linking



Layer 1: Shared-Infra (Foundation)

Provisions VNets, DNS Zones, and the Firewall.
Creates the physical boundary.

Layer 2: AKS / App-Core (Workloads)

Injects compute into the foundation.

Referential Integrity

We strictly avoid hardcoding resource IDs. App-Core consumes the outputs of Shared-Infra natively.

Passing `subnet_id = azurerm_subnet.id` upward creates an implicit dependency graph ensuring Terraform calculates the exact execution order mathematically, eliminating Chicken and Egg deployment failures.

Architectural Assessment: The Balance Sheet

✓ Architectural Dividends

- ✓ Zero collision risk between experimental and live infrastructure.
- ✓ Absolute Tier Parity (ENG identical to PRO).
- ✓ Highly secure, centralized egress via Hub Firewall.
- ✓ Dry codebase; rapid global expansion.

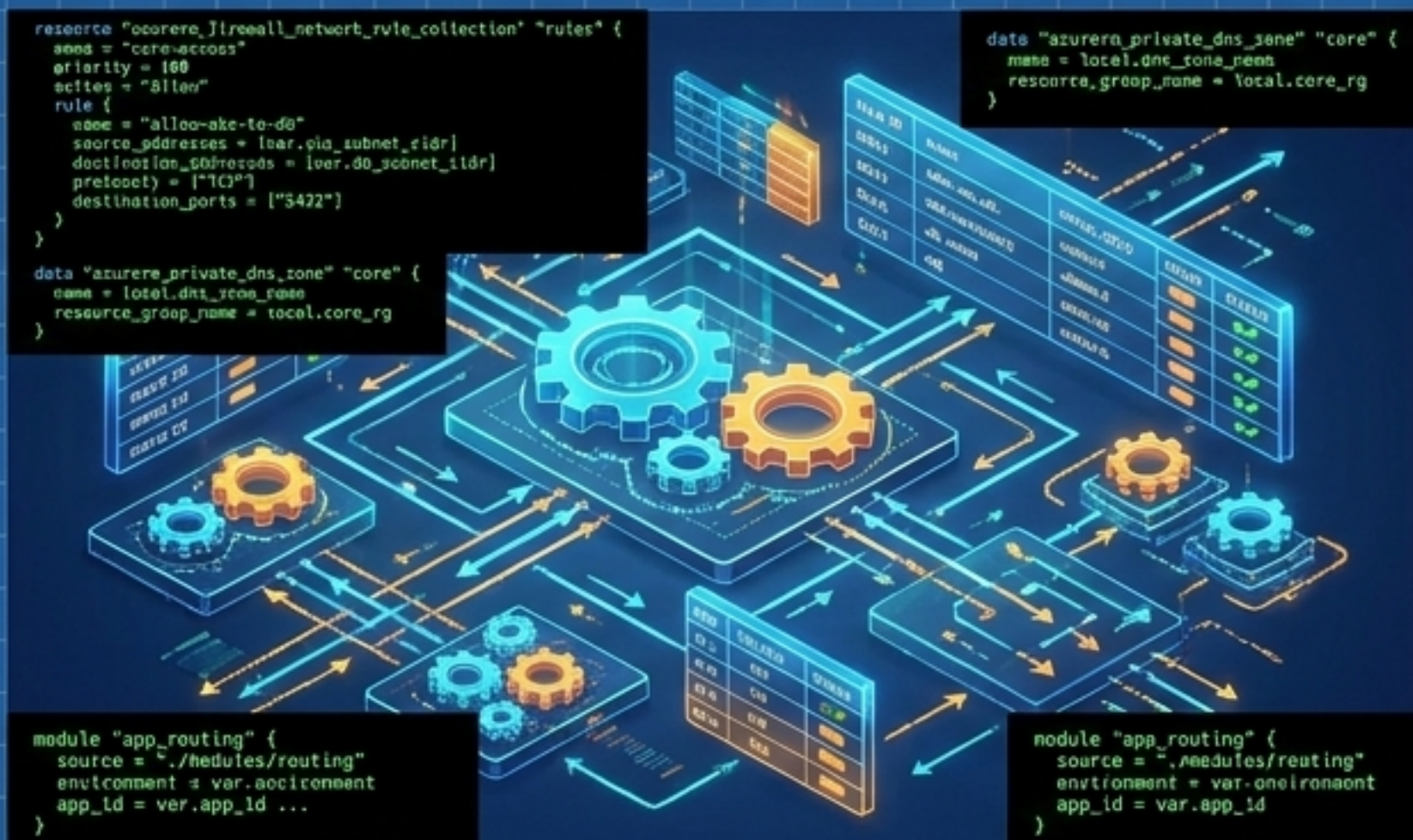
⚠ Operational Realities

- ⚠ Operationally hard to construct initially.
- ⚠ Steep learning curve for new engineers navigating the custom locals.tf engine.
- ⚠ Overhead of managing wildcard certificates for transient shadow environments.
- ⚠ Managing decoupled state across a federated multi-repo architecture requires strict pipeline discipline.

The Dynamic Blueprint

The Ultimate Output: Frictionless Developer Experience

Platform Engineering Layer



Application Developer View

Push to Deploy

The severe complexity of the networking, DNS generation, and Terraform logic is pushed entirely down into the Platform layer. Because the locals.tf engine handles all state tracking, identity wiring, and dynamic domain routing, developers simply commit their code. The infrastructure automatically catches it, isolates it, secures it, and routes to it.

The true value of this design is invisible, secure velocity.

PROJECT: CLOUD_ARCHITECTURE_V2.3

DRAWING NO: F-ET-009

DATE: OCT 28 2024